

LP-V Distributed Systems – Viva Questions & Answers

Subject Code: 414454 | **Sem VIII** | D Y Patil College of Engineering, Akurdi

Assignment 1: Multi-threaded Client/Server using Java RMI

Q1. What is RMI (Remote Method Invocation)?

RMI is a Java mechanism that allows an object executing in one JVM to invoke methods on an object in another JVM (on a different machine). It makes distributed applications appear as a group of communicating objects, supporting multi-threaded client/server communication transparently.

Q2. What is RMI Registry?

RMI Registry is a bootstrap naming service that runs on the server host. The server binds remote objects to names in the registry using `Naming.rebind()`, and clients look up those names using `Naming.lookup()` to obtain a remote object reference.

Q3. What is a Stub in RMI?

A Stub is a client-side proxy object that presents the same interface as the remote server. When a method is called on the stub, it:

1. Initiates a connection to the remote JVM.
2. Marshals (serializes) the parameters and sends them over the network.
3. Waits for the result, unmarshals it, and returns it to the caller.

Q4. What is a Skeleton in RMI?

A Skeleton is the server-side counterpart to the stub. It:

1. Receives the incoming call from the stub.
2. Unmarshals the parameters.
3. Invokes the actual remote object method.
4. Marshals and returns the result back to the stub.

Q5. What is marshalling and unmarshalling in RMI?

- **Marshalling:** Serializing Java objects/parameters so they can be sent over a network.

- **Unmarshalling:** Deserializing the received byte stream back into Java objects on the remote machine.

Q6. What is `UnicastRemoteObject` in RMI?

`UnicastRemoteObject` is a base class that all remote object implementations must extend. It provides the infrastructure to make objects remotely accessible – registering the object with the RMI runtime so it can receive remote calls.

Q7. What is `RemoteException` in RMI?

`RemoteException` is a checked exception that every remote method must declare in its `throws` clause. It wraps network failures, marshaling errors, and other remote communication problems.

Q8. What is the `Naming` class in RMI?

The `Naming` class (in `java.rmi`) provides the methods to interact with the registry:

- `Naming.rebind("Server", obj)` – Registers the remote object on the server.
- `Naming.lookup("rmi://localhost/Server")` – Returns a reference to the remote object on the client.

Q9. What command starts the RMI Registry?

```
start rmiregistry
```

This starts the RMI naming service on port 1099 (default) on the server machine.

Q10. What is the difference between RMI and regular method calls?

In a regular method call, both caller and callee are in the same JVM. In RMI, they are in different JVMs (possibly on different machines), and the call involves network communication, serialization, and stub/skeleton proxies.

Q11. How does multi-threading manifest in an RMI server?

The RMI runtime dispatches each client call in a separate thread. Multiple clients can simultaneously invoke remote methods, making RMI servers inherently multi-threaded. Shared state in `ServerImpl` must be synchronized.

Q12. What files are involved in the RMI assignment?

- `ServerIntf.java` – Remote interface (`extends Remote`)
- `ServerImpl.java` – Implementation (`extends UnicastRemoteObject implements ServerIntf`)
- `Server.java` – Registers the implementation with the registry
- `Client.java` – Looks up the remote object and calls its methods

Q13. What operations does the RMI assignment implement?

Addition, subtraction, multiplication, division, square, and square root of two numbers — all executed on the server but triggered remotely by the client.

Assignment 2: Distributed Application using CORBA (String Reversal / Calculator)

Q14. What is CORBA?

CORBA (Common Object Request Broker Architecture) is an open-standard, vendor-independent middleware developed by the Object Management Group (OMG). It allows objects written in different programming languages and running on different operating systems to communicate over a network.

Q15. What is ORB (Object Request Broker)?

ORB is the core middleware component of CORBA. It manages object references, routes client requests to the correct server object, handles marshalling of arguments, and uses IIOP for network communication.

Q16. What is IIOP?

IIOP (Internet Inter-ORB Protocol) is the standard wire protocol used by CORBA objects to communicate across networks and between different ORB implementations. It runs over TCP/IP.

Q17. What is IDL in CORBA?

IDL (Interface Definition Language) is a language-neutral syntax for defining the interface (methods, parameters, return types) of a remote CORBA object. The `idlj` compiler translates an `.idl` file into Java stubs and skeletons.

Q18. What command is used to compile the IDL file?

```
idlj -fall ReverseModule.idl
```

The `-fall` flag generates both client-side stubs and server-side skeletons.

Q19. What files does `idlj` generate from `ReverseModule.idl`?

Inside the `ReverseModule/` folder:

- `_ReverseStub.java` — Client-side stub
- `ReversePOA.java` — Server-side skeleton (abstract class)
- `Reverse.java` — Java interface
- `ReverseHelper.java` — Provides `narrow()` for casting object references
- `ReverseHolder.java` — Used for `out` / `inout` IDL parameters

- `ReverseOperations.java` – Method signatures shared by stub and skeleton

Q20. What is POA (Portable Object Adapter)?

POA (Portable Object Adapter) connects a client request (via an object reference) to the correct server-side implementation. It makes CORBA object implementations portable across different ORB products.

Q21. What is ORBD and how is it started?

ORBD (Object Request Broker Daemon) is a bootstrap naming service provided with the JDK. It allows clients to transparently locate persistent CORBA objects.

```
orbd -ORBInitialPort 1050
```

Q22. What is the sequence of commands to run the CORBA assignment?

```
# 1. Generate stubs/skeletons from IDL
idlj -fall ReverseModule.idl

# 2. Compile all Java files
javac *.java ReverseModule/*.java

# 3. Start ORBD (naming service)
orbd -ORBInitialPort 1050

# 4. Start the server
java ReverseServer -ORBInitialPort 1050 -ORBInitialHost localhost

# 5. Run the client
java ReverseClient -ORBInitialPort 1050 -ORBInitialHost localhost
```

Q23. What is the difference between CORBA and RMI?

Feature	CORBA	RMI
Language	Language-neutral (Java, C++, etc.)	Java only
Interface	Defined in IDL	Defined as Java interface
Object passing	By reference only	By value (serialization) or reference
Protocol	IIOP	Java-proprietary
Complexity	Higher	Lower (within Java)

Q24. What is a Naming Service in CORBA?

A naming service binds a name string to a CORBA object reference. The server registers its object with a name, and clients look up that name to get the reference and invoke methods on it.

Assignment 3: Distributed Sum of Array Elements using MPI / OpenMP

Q25. What is MPI (Message Passing Interface)?

MPI is a standardized API for communication between processes running on different processors/machines. It is the most widely used model for distributed-memory parallel programming. OpenMPI and MPICH are popular implementations.

Q26. What is the key idea of the array sum assignment?

An array of N elements is divided into N/n sub-arrays and distributed to n processors. Each processor computes the partial (intermediate) sum of its share, then all partial sums are gathered at the root process to compute the total sum.

Q27. What is `MPI_COMM_WORLD` ?

`MPI_COMM_WORLD` is the default communicator that contains all processes started by `mpirun` . It provides the context for all collective and point-to-point communication.

Q28. What is the rank of a process in MPI?

The rank is a unique integer ID automatically assigned to each process within a communicator, starting from 0. It is used to identify which process executes which part of the program.

Q29. What does `MPI_Comm_size` return?

```
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

Returns the total number of processes in the communicator, stored in `size` .

Q30. What is `MPI_Scatter` ?

`MPI_Scatter` is a collective operation that partitions an array at the root process and distributes one unique chunk to each process. Each process receives N/n elements.

Q31. What is `MPI_Reduce` ?

`MPI_Reduce` collects values from all processes using a specified operation (e.g., `MPI_SUM`) and stores the result at the root process. Used to sum all partial results into a final total.

Q32. What is the syntax of `MPI_Send` and `MPI_Recv` ?

```
MPI_Send(void *buf, int count, MPI_Datatype dtype, int dest, int tag, MPI_
MPI_Recv(void *buf, int count, MPI_Datatype dtype, int src, int tag, MPI_
```

Q33. What is the SPMD programming model?

Single Program Multiple Data (SPMD) – the same program runs on all processors simultaneously, but each process operates on a different portion of data. Controlled using `rank`.

Q34. What are `MPI_Init` and `MPI_Finalize`?

- `MPI_Init(&argc, &argv)` – Initializes the MPI environment. Must be the first MPI call.
- `MPI_Finalize()` – Cleans up all MPI state. Must be the last MPI call.

Q35. How do you compile and run the MPI program?

```
# Compile
mpicc Assignment3.c -o mpi_program

# Run with 4 processes
mpirun -np 4 ./mpi_program

# If "not enough slots" error:
mpirun --oversubscribe -np 4 ./mpi_program
```

Q36. What is OpenMP and how does it differ from MPI?

	MPI	OpenMP
Memory	Distributed memory	Shared memory
Processes	Multiple processes	Multiple threads
Communication	Explicit message passing	Implicit shared variables
Usage	Multi-machine clusters	Single multi-core machine

Assignment 4: Berkeley Algorithm for Clock Synchronization

Q37. What is Clock Synchronization in distributed systems?

In a distributed system, each node has an independent hardware clock. These clocks drift at different rates, causing inconsistencies. Clock synchronization algorithms adjust all clocks to agree on a common time, ensuring correct ordering of events.

Q38. What is the Berkeley Algorithm?

The Berkeley Algorithm is a distributed clock synchronization protocol where a master (daemon) periodically polls all slave nodes for their current clock times. The master:

1. Polls all slaves and collects their clock readings.
2. Calculates the average clock time (excluding outliers using fault tolerance).
3. Computes the time adjustment needed for each node.
4. Sends each slave the difference (delta) to adjust its clock – not the absolute time.

Q39. Why does the Berkeley Algorithm send a delta (difference) instead of absolute time?

Sending the absolute time would require an extra round-trip to account for network propagation delay. By computing and sending the difference, the algorithm eliminates the need for the master to have an accurate absolute time – only relative consistency matters.

Q40. What is the role of the Master (Server) in the Berkeley Algorithm?

The server (master node) in `server.py`:

1. Accepts connections from slave clients.
2. Receives each client's current timestamp.
3. Computes the average clock difference across all connected clients.
4. Sends each client a synchronized timestamp to adjust to.

Q41. What is the role of the Slave (Client) in the Berkeley Algorithm?

Each client (`client.py`):

1. Connects to the master server on port 8080.
2. Continuously sends its current local time every 5 seconds.
3. Receives the synchronized time from the master and displays it.

Q42. What Python libraries are used in the Berkeley assignment?

- `socket` – for TCP network communication
- `threading` – to handle multiple clients simultaneously and concurrent send/receive
- `datetime` – to capture current timestamps
- `dateutil.parser` – to parse datetime strings received over the socket

Q43. What is the difference between NTP and Berkeley Algorithm?

	NTP	Berkeley Algorithm
Time source	External UTC time servers	Master node (no external reference)
Direction	Slaves sync to accurate external source	Master takes average of all clocks
Accuracy	Very high	Moderate (internal consistency)
Use case	Internet-wide time sync	LAN without external time reference

Q44. What is clock drift?

Clock drift is the gradual divergence of a computer's clock from actual time, caused by slight

inaccuracies in the oscillator. Even a 1-second drift per day can cause serious inconsistencies in distributed systems.

Q45. How do you run the Berkeley Clock Sync assignment?

```
# Install dependency
pip install python-dateutil

# Terminal 1 – Start master server
python server.py

# Terminal 2, 3, ... – Start slave clients
python client.py
```

Multiple clients can be started simultaneously to simulate a distributed network.

Assignment 5: Token Ring Mutual Exclusion Algorithm

Q46. What is Mutual Exclusion in distributed systems?

Mutual exclusion ensures that only one process at a time can access a shared critical section (shared resource). In distributed systems, there is no shared memory, so algorithms use message passing to coordinate access.

Q47. What is the Token Ring Algorithm?

The Token Ring Algorithm uses a logical ring topology where a special message (the "token") circulates continuously around the ring. A process may enter the critical section **only if it holds the token**. After finishing, it passes the token to its successor.

Q48. What is the role of the Token in the algorithm?

The token is a permission slip. Possession of the token grants exclusive access to the critical section. Only one token exists in the ring at any time, guaranteeing mutual exclusion.

Q49. What is the step-by-step flow of the Token Ring assignment?

1. User enters the number of nodes (n) in the ring.
2. Token starts at Node 0.
3. User specifies sender, receiver, and data.
4. The token passes sequentially from the current holder to the sender node.
5. Once the sender holds the token, data is passed hop-by-hop to the receiver.
6. The receiver gets the data; token remains at the sender.
7. User can choose to send again.

Q50. How does the Token Ring algorithm achieve Mutual Exclusion?

- Only the process holding the token can transmit data (enter the critical section).
- Since there is exactly one token in the ring, only one process can be in the critical section at a time – satisfying mutual exclusion.

Q51. What are the advantages of Token Ring?

1. Starvation-free – every process eventually gets the token.
2. Fair – token passes in order around the ring.
3. No collisions – only one process transmits at a time.

Q52. What is a drawback of Token Ring?

If the process holding the token crashes, the token is lost and the entire ring halts. A token recovery mechanism is needed (detect missing token and regenerate it).

Q53. What is the difference between Token Ring and Bully Algorithm?

- **Token Ring:** Used for mutual exclusion (controlling access to a shared resource). Processes take turns.
- **Bully/Ring Algorithm:** Used for leader election (choosing a coordinator). One process is elected as the winner.

Q54. How do you compile and run the Token Ring assignment?

```
cd "d:\LLM-wiki\code\Assignment 5"
javac TokenRing.java
java TokenRing
```

Enter number of nodes, then specify sender, receiver, and data for each transmission.

Assignment 6: Bully and Ring Algorithm for Leader Election

Q55. Why are election algorithms needed in distributed systems?

Many distributed algorithms require a single process to act as a coordinator (leader). If the coordinator crashes, a new one must be elected. Election algorithms provide a systematic way to choose a new coordinator from the remaining active processes.

Q56. What is the Bully Algorithm?

The Bully Algorithm elects the process with the highest process ID/priority as coordinator:

1. A process P notices the coordinator has failed and sends an ELECTION message to all higher-priority processes.
2. If no higher process responds, P declares itself coordinator and sends a COORDINATOR message to all lower processes.
3. If a higher-priority process responds with OK, it takes over the election.
4. The process with the highest priority that is alive always wins – it "bullies" others.

Q57. Why is it called the "Bully" algorithm?

Because the highest-priority (biggest) process always wins and forces all lower-priority processes to accept it as the new coordinator – it bullies them into submission.

Q58. What is the Ring Algorithm for Election?

The Ring Algorithm uses a unidirectional logical ring:

1. A process notices the coordinator has failed and sends an ELECTION message with its own ID to its successor.
2. Each process that receives the message appends its ID and passes it to the next node.
3. When the message returns to the initiator, the process with the maximum ID in the list is declared the new coordinator.
4. A COORDINATOR message is sent around the ring to announce the winner.

Q59. What is the key difference between Bully and Ring algorithms?

Feature	Bully Algorithm	Ring Algorithm
Topology	Any (can contact any process)	Ring (messages travel in one direction)
Winner	Highest ID / priority	Maximum ID in circulated list
Messages	$O(n^2)$ worst case	$O(n)$ messages
Failure detection	Requires timeout	Requires timeout

Q60. What inputs does the Bully Algorithm simulation require?

1. Number of processes (n).
2. Status of each process: 1 = alive, 0 = dead/crashed.
3. Priority of each process (higher priority wins).
4. The process number that initiates the election.

Q61. What happens if an initiating process is dead in the Bully simulation?

The program validates the input. If the selected initiator has `status = 0` (dead), it alerts the user and asks them to select a live process before proceeding.

Q62. What happens if a crashed process recovers in the Bully Algorithm?

The recovered process holds a new election. If it has the highest priority, it sends ELECTION messages

to all and wins, becoming the new coordinator — regardless of who the current coordinator is.

Q63. What are the assumptions common to both election algorithms?

1. Each process has a unique ID / priority.
2. All processes know each other's IDs.
3. A process can detect coordinator failure (via timeout or heartbeat).

Q64. How do you run the election algorithm assignments?

```
cd "d:\LLM-wiki\code\Assignment 6"
javac BullyAlgoExample.java RingAlgorithm.java

# Run Bully Algorithm
java BullyAlgoExample

# Run Ring Algorithm
java RingAlgorithm
```

Assignment 7: Simple Web Service and Distributed Consumer Application

Q65. What is a Web Service?

A web service is a software component accessible over a network using standard protocols (HTTP, XML). It is platform-independent, language-neutral, and self-describing. Any application that speaks the protocol can consume it.

Q66. What are the two types of web services?

1. **SOAP (Simple Object Access Protocol):** XML-based protocol; uses WSDL for description; uses UDDI for discovery; more formal/complex; supports WS-Security.
2. **REST (Representational State Transfer):** Architectural style; uses HTTP methods (GET, POST, PUT, DELETE); resources identified by URIs; lighter and simpler; preferred for modern APIs.

Q67. What is WSDL?

WSDL (Web Services Description Language) is an XML document that describes a SOAP web service — what methods it exposes, the parameters they accept, the return types, and the endpoint URL. It acts as a contract between service provider and consumer.

Q68. What is UDDI?

UDDI (Universal Description, Discovery, and Integration) is an XML-based directory/registry where web services are published so that clients can discover them. It acts like a "phone book" for web services.

Q69. What are the three roles in a web service architecture?

1. **Service Provider:** Implements and publishes the web service.
2. **Service Requestor (Client):** Discovers and invokes the web service.
3. **Service Registry (UDDI):** Directory where services are registered and discovered.

Q70. What is JAX-WS?

JAX-WS (Java API for XML Web Services) is the standard Java API for building SOAP web services. The `@WebService` annotation marks a class as a web service endpoint, and JAX-WS automatically generates the WSDL.

Q71. How does a SOAP request-response cycle work?

1. Client prepares request as a SOAP XML message.
2. Client sends it as an HTTP POST request to the service endpoint.
3. The service parses the SOAP envelope and calls the appropriate method.
4. The result is wrapped in a SOAP response XML and returned.
5. The client parses the SOAP response and uses the result.

Q72. What tools are used for the Web Services assignment (Assignment 7)?

- **NetBeans IDE** – to create and deploy the web service project
- **GlassFish Server** – application server that hosts the SOAP web service
- **JDK 8** – required for SOAP/JAX-WS support in NetBeans

Q73. What is the step-by-step process to create a SOAP web service in NetBeans?

1. `File → New Project → Java Web → Web Application` (name: `Assgn_7`)
2. Right-click project → `New → Web Service` (name: `mycalculator`, package: `com.mysevice`)
3. Right-click `mycalculator` under Web Services → `Add Operation` (e.g., add, subtract, multiply, divide)
4. Right-click project → `Build` then `Deploy`
5. Right-click `mycalculator` → `Test Web Service` → browser opens WSDL tester

Q74. How is the web service client (consumer) created?

1. Create a new Web Application project (`CalculatorWebService_Client`)
2. `New → Web Service Client` – paste the WSDL URL from the server project
3. Create `index.html` with a form for input values
4. Create a Servlet (action target of the form)
5. Drag service methods from the Web Service References panel into the Servlet
6. Parse input, call the remote method, and write the result to the response
7. Clean and Build, then run `index.html`

Q75. What is the difference between SOAP and REST?

Feature	SOAP	REST
Type	Protocol	Architectural style
Format	XML only	XML, JSON, etc.
Transport	HTTP, SMTP, etc.	HTTP only
Security	WS-Security standard	HTTPS
Performance	Heavier (XML overhead)	Lighter
Tooling	WSDL + UDDI	No formal standard

General Distributed Systems Concepts

Q76. What is a Distributed System?

A distributed system is a collection of independent computers that appears to its users as a single coherent system. Processes on different machines communicate and coordinate over a network to achieve a common goal.

Q77. What is middleware in distributed systems?

Middleware is a software layer between the distributed application and the operating system/network. It provides standard interfaces that enable applications to use networked resources and services, alleviating heterogeneity and enabling interoperability. Examples: CORBA ORB, RMI runtime, JMS broker.

Q78. What is transparency in distributed systems?

Transparency hides the complexity of distribution from users/developers:

- **Location transparency** – hide where a resource is located
- **Replication transparency** – hide that multiple copies exist
- **Failure transparency** – hide partial failures
- **Migration transparency** – hide that a resource may move
- **Concurrency transparency** – hide that multiple clients share a resource

Q79. What is heterogeneity in distributed systems?

Heterogeneity refers to differences in hardware, operating systems, programming languages, or network protocols among components. Middleware (like CORBA, RMI) overcomes these differences by providing a common communication layer.

Q80. What is scalability in distributed systems?

A system is scalable if it can grow in size (more users, more data, more machines) without loss of

performance. Distributed systems achieve scalability by adding more nodes rather than upgrading a single machine (horizontal scaling).

Q81. What is a critical section?

A critical section is a segment of code that accesses a shared resource (e.g., a file, memory region, database) and must not be executed by more than one process at a time. Mutual exclusion algorithms (Token Ring) control access to critical sections.

Q82. What is the difference between synchronous and asynchronous communication in distributed systems?

- **Synchronous:** Sender blocks and waits until the receiver processes the message and replies (e.g., RMI calls).
- **Asynchronous:** Sender sends and continues execution without waiting. The reply (if any) arrives later (e.g., message queues, event-driven systems).

End of Viva Questions – LP-V Distributed Systems